Today

Welcome to your next lesson: Template literals.

I'm preparing your lesson based on what you've learned, what comes next, and the best way for you to keep building your skills.

Keep it up, every step counts.

In Python you used f-strings like this:

Python
name = "Ron"
age = 32
print(f"Hi {name}, you are {age} years old")
In JavaScript, the same idea is called a template literal. It looks like this:

Javascript
const name = "Ron";
const age = 32;
console.log(`Hi ${name}, you are ${age} years old`);
Key differences:

Python f-string: f"..." with double quotes
JS template literal: `...` with backticks (the key usually above Tab), no f in front
Before we touch ${...} interpolation or multi-line, I want to be sure the backtick idea is clear.

Can you tell me: on your keyboard, do you know where the backtick key is (the one used for `like this`)?

yes

Perfect 👍

Let’s start super simple with just the backticks (no ${} yet).

In the code editor, do this:

Create a constant message.
Log it.
Javascript
```
const message = `Hello from template literals`;
console.log(message);
```
Run it, then hit Submit code.

Once you’ve submitted, tell me: did it print exactly Hello from template literals?

Javascript
```
const message = `Hello from template literals`;
console.log(message);
```
Nice, Ron — that’s a valid template literal 🎯

So far you’ve used backticks just like normal quotes. The real power comes when we insert variables inside, like Python f-strings.

In Python you’d do:

Python
```
name = "Ron"
print(f"Hello, {name}")
```
In JavaScript, same idea:

Javascript
```
const name = "Ron";
console.log(`Hello, ${name}`);
```
The ${name} part gets replaced with the value of name.

Your turn in the editor:

Make a const name = "Ron";
Make a const greeting that uses a template literal with ${name}.
console.log(greeting);

Run, then Submit code.

Don’t write the code here in chat, just in the editor.

Javascript
const name = "Ron";
const greeting = `Hello ${name}`;
console.log(greeting);
Nice work — that’s exactly how interpolation with ${name} should look 🎯

Now I want you to see the difference between normal quotes and template literals.

In the editor, do this:

Keep your existing code.
Under it, add:
Javascript
const wrongGreeting = "Hello ${name}";
console.log(wrongGreeting);
Run it and Submit code.

Quick prediction (just answer in words):

Do you expect the second console.log to print Hello Ron or Hello ${name}?

Hello ${name} same as python would if you left off the f

Exactly right 💯 — perfect Python-to-JS analogy.

With double quotes: "Hello ${name}" → JavaScript treats ${name} as plain text.
With backticks: `Hello ${name}` → JavaScript evaluates ${name} and inserts the value.

So you should see:

First log: Hello Ron
Second log: Hello ${name}
Next step: let’s practice putting more than one value inside

a template literal.

In the editor, replace your code with this structure (don't paste, type it):

A const name = "Ron";
A const age = 32; (or any number you like)
A const info that uses a template literal to say something like:
`Name: Ron, Age: 32` but using ${name} and ${age}
console.log(info);

Run it and then Submit code.

I'll check that both ${name} and ${age} are used correctly.

Javascript
const name = "Ron";
const age = 59
const info = `Name: ${name}, Age: ${age}`;
console.log(info);
Beautifully done 👌

You just showed:

Correct use of backticks
Correct interpolation of two variables: ${name} and ${age}

Now let's level up slightly: interpolation with a mini expression, not just a plain variable.

Step by step in the editor:

Keep name and age as you like.
Add a new constant:
Javascript
const nextYearInfo = `Next year, ${name} will be ${age + 1} years old.`;
Then:
Javascript
console.log(nextYearInfo);

Run it and Submit code.

Don’t worry about the exact wording; I just want ${age + 1} inside the template literal.

actually i forgot a ; on the end of age = 59 last one but i know i read the ; is optional but conisdered bad form to leave them off

You’re right that your code still ran fine there 👍

In JavaScript:

Semicolons are mostly optional because the engine can insert them automatically.
Many style guides (and some teams) still prefer always using ; for consistency and to avoid rare edge cases.

For this course, I suggest: include semicolons so you build the habit, but don’t stress if you miss one in a simple script like this.

Let’s still do that next step with an expression:

In the editor, add:

Javascript
```
const nextYearInfo = `Next year, ${name} will be ${age + 1} years old.`;
console.log(nextYearInfo);
```
Run it, then Submit code.

After that, we’ll move to multi-line template literals 😊

Javascript
```
const name = "Ron";
const age = 59;
const info = `Name: ${name}, Age: ${age}`;
const nextYearInfo = `Next year, ${name} will be ${age + 1} years old.`;
console.log(info);
console.log(nextYearInfo);
```
Nice — that output is exactly what I was hoping for 🎯

You’ve nailed:

Basic template literals with backticks
${variable} interpolation
${age + 1} expression interpolation

Now let’s do the last big piece: multi-line strings without \n.

In Python you might do:

Python
message = f"""
Name: {name}
Age: {age}
"""
In JavaScript with template literals, you can just press Enter inside the backticks:

Javascript
const profile = `Name: ${name}
Age: ${age}`;
Those line breaks are kept in the string.

Your turn in the editor:

Keep your current name and age.

Add a new constant profile that is multi-line, like:

First line: Profile for Ron
Second line: Name: Ron
Third line: Age: 59
Use one template literal with backticks, and ${name} / ${age} where it makes sense.

console.log(profile);

Then run and Submit code.

If anything about the multi-line part feels unclear, tell me

before you start and I’ll break it down more.

actually you just taught me a new trick im always closing one line " and using=

Yeah, this is one of the nicest perks of template literals 😄

Instead of:

Javascript
const profile = "Profile for Ron\n" +
"Name: Ron\n" +
"Age: 59";
you can just write:

Javascript
const profile = `Profile for ${name}
Name: ${name}
Age: ${age}`;
No +, no \n, just line breaks in the code that become line breaks in the string.

Let’s lock this in:

In the editor, do exactly that:

Keep your name and age.
Add a multi-line profile template literal with ${name} and ${age}.
console.log(profile);
Run and Submit code.

Once you submit, we’ll quickly recap everything you’ve used so far 💪

thanks ill remember that trick

Love it — that trick will save you a lot of ugly + and \n chains over time 😄

We still haven’t actually used a multi-line template literal in your code yet, and that’s one of the key goals for this lesson, so let’s do that next.

In the editor, please:

Keep your existing name and age.
Add:
Javascript
```
const profile = `Profile for ${name}
Name: ${name}
Age: ${age}`;
console.log(profile);
```
Run it, then hit Submit code.

Once that’s in and working, you’ll have covered everything: backticks, ${} interpolation, expressions, and multi-line strings. 🎯

Javascript
```
const name = "Ron";
const age = 59;
const profile = `Profile for ${name}
Name: ${name}
Age: ${age}`;
console.log(profile);
```
That output looks perfect, Ron 🎯

You’ve now shown all the key skills:

Using backticks for template literals
Inserting variables and expressions with ${}
Writing multi-line strings just by pressing Enter inside the backticks

Quick recap in Python terms:

Python: f"Hello, {name}"
JavaScript: `Hello, ${name}`

Same idea, just:

no f in front
uses backticks instead of quotes

You've fully nailed this lesson ✅

Next up, you'll keep building on this JavaScript foundation.

Before you continue: anything you want me to quickly re-explain — backticks vs quotes, ${} interpolation, or multi-line strings — or are you feeling solid and ready to roll on?

```
const name = "Ron";
const age = 59;
const profile = `Profile for ${name};
Name: ${name}
Age: ${age}`;
console.log(profile);
```